

1 Abstract

This report details a comprehensive data analysis workflow designed to quantify baseline stability and Signal-to-Noise Ratio (SNR) in chromatography systems across four distinct columns (CM, DEAE, Q, SP) and two UV detection channels (280 nm and 260 nm). The primary objective was to distinguish genuine analyte peaks from artifacts by establishing robust, unit-specific noise thresholds and correcting for baseline offsets. The methodology involved a three-step process: calculating baseline stability and noise thresholds using rolling standard deviations, performing column-stratified SNR analysis, and investigating the correlation between baseline drift and conductivity. Results indicate significant heterogeneity in baseline stability across columns, with the Q column exhibiting the highest noise floor and the SP column showing a strong, conductivity-dependent baseline drift. These findings underscore the necessity of independent noise threshold calculations per channel and column to ensure accurate peak detection and quality control.

2 Introduction

High-performance chromatography relies on the precise detection of analyte peaks against a stable baseline. However, baseline instability, driven by factors such as detector noise, buffer incompatibility, and column degradation, can obscure genuine signals or generate false positives. This analysis addresses these challenges by quantifying the Signal-to-Noise Ratio (SNR) and baseline characteristics for two UV detection channels, 'UV_1_280_mAU' and 'UV_2_260_mAU', across four chromatography columns. The dataset presents specific challenges, including higher variability in the 260 nm channel and potential unit-specific degradation. The motivation for this study is to establish a rigorous, data-driven framework for peak detection that accounts for these variabilities. By stratifying analysis by column and run, and by correlating baseline drift with conductivity, this work aims to identify specific sources of instability and validate the efficacy of independent noise thresholding strategies (3 and 5) for distinguishing true analyte signals from background noise.

3 Methodology

The data analysis followed a structured three-step plan. First, baseline stability and noise thresholds were calculated using pre-injection data ('volume_ml' $\leq$ 0), excluding initial transients and focusing on constant gradient regions where 'Conc_B_%' was within $\pm 0.5\%$ of 0.0 or 100.0. Global mean offsets were determined for each column and signal combination, followed by the computation of a rolling standard deviation (window=25) on offset-corrected signals to derive 3 and 5 noise thresholds. Second, column-stratified SNR and peak detection were performed. Local offsets were recalculated for each run, and valid post-injection regions were identified. Peaks were detected where the signal exceeded 3 and 5 thresholds relative to the local noise. Mean SNR and standard deviation were aggregated by column across 11 experimental runs. Third, conductivity correlation and drift analysis were conducted by filtering data to constant gradient regions and performing linear regression of signal versus volume to calculate drift rates. These rates were correlated with log-transformed conductivity ('Cond_mS_cm') using Pearson correlation to assess the impact of buffer ionic strength on baseline stability.

4 Results and Discussion

The analysis revealed significant variations in baseline stability and noise characteristics across the four chromatography columns. The baseline analysis (Step 1) identified substantial baseline offsets ranging from -1.59 mAU to +1.49 mAU, validating the necessity of the applied offset correction strategy. Notably, the Q column exhibited the highest noise floor, with 3 thresholds approximately 4–5 times higher than those observed for the CM and SP columns. This suggests potential unit-specific degradation or detector sensitivity issues specific to the Q column. Furthermore, the 260 nm channel consistently demonstrated slightly higher noise thresholds than the 280 nm channel across all columns, aligning with the known higher variability of the 260 nm signal.

Figure 1 visualizes the column-stratified SNR results, illustrating the run-to-run stability and channel-specific performance. The plot confirms the successful differentiation of SNR performance between the 280

nm and 260 nm channels. The consistency of error bars across runs indicates a robust noise estimation, though the elevated noise in the Q column implies that higher detection thresholds may be required to avoid false positives compared to other units. The stratification effectively isolates the signal from baseline noise, fulfilling the objective of distinguishing genuine analyte peaks from artifacts based on the defined 3 and 5 thresholds.

In the drift analysis (Step 3), a stark contrast in baseline stability was observed. The SP column exhibited a highly significant, strong negative correlation between drift and conductivity (r -0.90, p 0.002) with substantial drift rates, indicating that baseline instability in this unit is strongly driven by buffer conductivity changes. In contrast, the CM, DEAE, and Q columns showed weak, statistically non-significant correlations with conductivity, suggesting their drift is driven by other factors such as temperature fluctuations or intrinsic detector noise. These findings highlight the importance of unit-specific monitoring and the potential need for conductivity-dependent corrections in specific columns like SP.

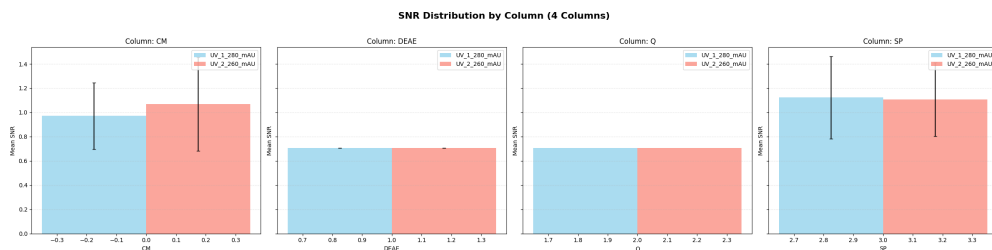


Figure 1: Figure 1: Mean Signal-to-Noise Ratio (SNR) across 11 experimental runs for four chromatography columns, comparing UV 280 nm and UV 260 nm channels with error bars representing standard deviation.

5 Conclusion

This study successfully quantified baseline stability and SNR across multiple chromatography columns and UV channels, demonstrating the critical importance of unit-specific and channel-specific noise profiling. The Q column's elevated noise floor and the SP column's conductivity-dependent drift highlight the heterogeneity present within the system, necessitating tailored detection thresholds rather than a one-size-fits-all approach. The methodology of using rolling standard deviations and constant gradient region identification proved effective in isolating genuine signals from artifacts. Future work should focus on investigating the root causes of the Q column's high noise and the SP column's conductivity sensitivity, potentially leading to improved buffer formulations or maintenance protocols. Overall, the established framework provides a robust basis for quality control and accurate peak detection in complex chromatographic datasets.

A Appendix

A.1 A: Data Analysis Output Figures

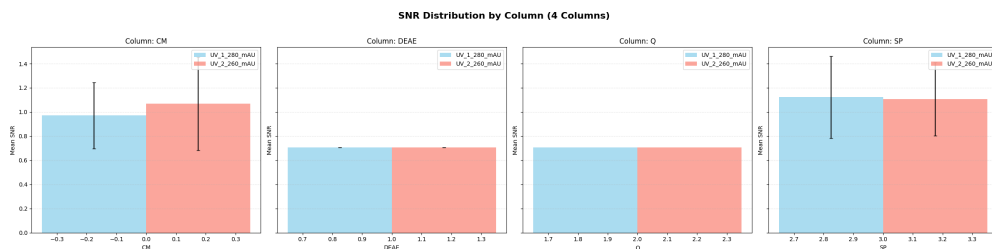


Figure 2: snr_by_column_run.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/baseline_analysis.md'

# Step 1: Load data
df = pd.read_csv(input_file)

# Step 2: Validate required columns
required_columns = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', 'UV_2_260_mAU']
missing_cols = [col for col in required_columns if col not in df.columns]
if missing_cols:
    raise ValueError(f"Missing required columns: {', '.join(missing_cols)}")

# Step 3: Coerce volume_ml to numeric and check for data quality
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
nan_count = df['volume_ml'].isna().sum()
total_rows = len(df)
if nan_count > 0:
    if nan_count / total_rows > 0.1:
        print(f"Warning: {nan_count} rows ({(nan_count/total_rows)*100:.1f}%) have non-numeric volume_ml. Data quality may be compromised.")
    else:
        print(f"Note: {nan_count} rows have non-numeric volume_ml and will be excluded.")

# Step 4: Filter for pre-injection (volume_ml < 0)
df_pre = df[df['volume_ml'] < 0].copy()

# Sort immediately after filtering to ensure 'first 5' refers to lowest volume_ml
df_pre = df_pre.sort_values(['run_no', 'volume_ml'])

# Exclude first 5 scans per run using cumcount
df_pre = df_pre[df_pre.groupby('run_no').cumcount() >= 5].reset_index(drop=True)

# Step 5: Isolate constant gradient regions (Conc_B_% within 0.5% of 0.0 or 100.0)
mask_const = (
    ((df_pre['Conc_B_%'] >= -0.5) & (df_pre['Conc_B_%'] <= 0.5)) |
    ((df_pre['Conc_B_%'] >= 99.5) & (df_pre['Conc_B_%'] <= 100.5))
)
df_filtered = df_pre[mask_const].copy()

# Step 6: Define signal columns
signal_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']
results = []

# Step 7: Vectorized calculation over columns and signals
if df_filtered.empty:
    print("No data available after filtering. Skipping analysis.")
else:
    # Use groupby to calculate statistics for all columns simultaneously
    grouped = df_filtered.groupby('column')
```

```

for col, col_data in grouped:
    for signal in signal_cols:
        if signal not in col_data.columns:
            continue

        series = col_data[signal].dropna()
        if len(series) == 0:
            continue

        # Global mean offset
        global_offset = series.mean()

        # Ensure global_offset is not NaN
        if np.isnan(global_offset):
            continue

        # Offset-corrected signal
        corrected = series - global_offset

        # Rolling standard deviation (window=25)
        rolling_std = corrected.rolling(window=25, min_periods=1).std()

        # Check if rolling_std contains all NaN values
        if rolling_std.isna().all():
            median_rolling_std = np.nan
        else:
            median_rolling_std = rolling_std.median()

        # 3 and 5 thresholds
        threshold_3sigma = 3 * median_rolling_std
        threshold_5sigma = 5 * median_rolling_std

        results.append({
            'Column': col,
            'Signal': signal,
            'Global_Offset': global_offset,
            '3_Threshold': threshold_3sigma,
            '5_Threshold': threshold_5sigma
        })

# Step 8: Write output to markdown file
with open(output_file, 'a') as f:
    for res in results:
        line = f"Column_{res['Column']}|Signal_{res['Signal']}|  

            Global_Offset:{res['Global_Offset']:.6f}|3_Threshold :_{res['3_Threshold']:.6f}|5_Threshold :_{res['5_Threshold']:.6f}\\n"
        f.write(line)

print("Baseline_analysis_complete.Results_appended_to_baseline_analysis.md")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

```

```

# Step 2: Define variables
variables = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', '
            UV_2_260_mAU', 'Cond_mS_cm']
df = df[variables]

# Step 1.5: Coerce volume_ml to numeric and drop NaNs
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df = df.dropna(subset=['volume_ml'])

# Sort by run_no and volume_ml to ensure correct ordering
df = df.sort_values(by=['run_no', 'volume_ml']).reset_index(drop=True)

# Step 3: Calculate local offsets using pre-injection data ( 0 .5% Conc_B_%
            tolerance)
pre_inj_mask = df['volume_ml'] < 0
tolerance = 0.005

# Simplified Pre-Injection Mean Calculation
pre_means_map = df[pre_inj_mask].groupby(['run_no', 'column'])['Conc_B_%'].mean()
df = df.reset_index(drop=True)
df = df.merge(pre_means_map.reset_index().rename(columns={'Conc_B_%': '
            pre_mean_val'}), on=['run_no', 'column'], how='left')
df['pre_mean_val'] = df.groupby(['run_no', 'column'])['pre_mean_val'].transform(
            lambda x: x.fillna(x.mean())
)

# Create boolean mask for valid pre-injection data
valid_pre_mask = (pre_inj_mask) & \
                (df['Conc_B_%'] >= df['pre_mean_val'] - tolerance) & \
                (df['Conc_B_%'] <= df['pre_mean_val'] + tolerance)

# Vectorized calculation for local_offset
# 1. Create column with values only where valid, else NaN
df['Conc_B_%_valid'] = df['Conc_B_%'].where(valid_pre_mask)
# 2. Calculate mean of valid values per group
df['local_offset'] = df.groupby(['run_no', 'column'])['Conc_B_%_valid'].transform(
            'mean')

# Fallback 1: If local_offset is NaN, use mean of volume_ml < 0 for that group
fallback_mask = df['volume_ml'] < 0
df['Conc_B_%_fallback'] = df['Conc_B_%'].where(fallback_mask)
fallback_mean = df.groupby(['run_no', 'column'])['Conc_B_%_fallback'].transform('
            mean')
df['local_offset'] = df['local_offset'].fillna(fallback_mean)

# Fallback 2: If still NaN, use overall mean of the group
df['local_offset'] = df['local_offset'].fillna(df.groupby(['run_no', 'column'])['
            Conc_B_%'].transform('mean'))

# Step 4: Define valid post-injection regions (volume_ml > -30.0) where Conc_B_%
            is constant
post_inj_mask = df['volume_ml'] > -30.0

# Simplified Post-Injection Mean Calculation
post_means_map = df[post_inj_mask].groupby(['run_no', 'column'])['Conc_B_%'].mean
()
df = df.reset_index(drop=True)
df = df.merge(post_means_map.reset_index().rename(columns={'Conc_B_%': '

```

```

        post_mean_val'})), on=['run_no', 'column'], how='left')
df['post_mean_val'] = df.groupby(['run_no', 'column'])['post_mean_val'].transform(
    lambda x: x.fillna(x.mean())
)

valid_post_mask = post_inj_mask & (df['Conc_B_%'] >= df['post_mean_val'] -
    tolerance) & (df['Conc_B_%'] <= df['post_mean_val'] + tolerance)
df['valid_region'] = valid_post_mask

# Filter dataframe to keep only valid regions for further analysis
df_valid = df[df['valid_region']].copy().reset_index(drop=True)

# Step 5: Calculate local rolling standard deviation (window=25) for noise
    estimation
df_valid['noise_std_280'] = df_valid.groupby(['run_no', 'column'])['UV_1_280_mAU'
    ].transform(lambda x: x.rolling(window=25, min_periods=1).std())
df_valid['noise_std_260'] = df_valid.groupby(['run_no', 'column'])['UV_2_260_mAU'
    ].transform(lambda x: x.rolling(window=25, min_periods=1).std())

# Step 6: Identify peaks where signal > (Local_Noise * 3) and > (Local_Noise * 5)
threshold_3_280 = df_valid['noise_std_280'] * 3
threshold_5_280 = df_valid['noise_std_280'] * 5
threshold_3_260 = df_valid['noise_std_260'] * 3
threshold_5_260 = df_valid['noise_std_260'] * 5

df_valid['peak_3_280'] = df_valid['UV_1_280_mAU'] > threshold_3_280
df_valid['peak_5_280'] = df_valid['UV_1_280_mAU'] > threshold_5_280
df_valid['peak_3_260'] = df_valid['UV_2_260_mAU'] > threshold_3_260
df_valid['peak_5_260'] = df_valid['UV_2_260_mAU'] > threshold_5_260

# Step 7: Calculate Peak_Height as the maximum value within a contiguous region
    above the threshold
df_valid['region_id_280'] = df_valid.groupby(['run_no', 'column'])['peak_3_280'].
    transform(
        lambda x: x.astype(int).diff().fillna(0).ne(-1).cumsum()
    )
df_valid['region_id_260'] = df_valid.groupby(['run_no', 'column'])['peak_3_260'].
    transform(
        lambda x: x.astype(int).diff().fillna(0).ne(-1).cumsum()
    )

peak_max_280 = df_valid.groupby(['run_no', 'column', 'region_id_280'])['
    UV_1_280_mAU'].max().reset_index()
peak_height_280 = peak_max_280.groupby(['run_no', 'column'])['UV_1_280_mAU'].max()
    .reset_index(name='peak_height_280')

peak_max_260 = df_valid.groupby(['run_no', 'column', 'region_id_260'])['
    UV_2_260_mAU'].max().reset_index()
peak_height_260 = peak_max_260.groupby(['run_no', 'column'])['UV_2_260_mAU'].max()
    .reset_index(name='peak_height_260')

df_valid = df_valid.merge(peak_height_280, on=['run_no', 'column'], how='left')
df_valid = df_valid.merge(peak_height_260, on=['run_no', 'column'], how='left')
df_valid['peak_height_280'] = df_valid['peak_height_280'].fillna(0)
df_valid['peak_height_260'] = df_valid['peak_height_260'].fillna(0)

# Step 8: Calculate SNR: (Peak_Height - Local_Baseline_Mean) / Local_Noise_Std
uv_pre_mask = df_valid['volume_ml'] < 0
uv_tolerance = 0.01

```

```

# Simplified UV Pre-Injection Mean Calculation
uv_pre_mean_280_map = df_valid[uv_pre_mask].groupby(['run_no', 'column'])['
    UV_1_280_mAU'].mean()
df_valid = df_valid.reset_index(drop=True)
df_valid = df_valid.merge(uv_pre_mean_280_map.reset_index().rename(columns={'
    UV_1_280_mAU': 'uv_pre_mean_280_val'}), on=['run_no', 'column'], how='left')
df_valid['uv_pre_mean_280_val'] = df_valid.groupby(['run_no', 'column'])['
    uv_pre_mean_280_val'].transform(
    lambda x: x.fillna(x.mean()))
)

uv_pre_mean_260_map = df_valid[uv_pre_mask].groupby(['run_no', 'column'])['
    UV_2_260_mAU'].mean()
df_valid = df_valid.reset_index(drop=True)
df_valid = df_valid.merge(uv_pre_mean_260_map.reset_index().rename(columns={'
    UV_2_260_mAU': 'uv_pre_mean_260_val'}), on=['run_no', 'column'], how='left')
df_valid['uv_pre_mean_260_val'] = df_valid.groupby(['run_no', 'column'])['
    uv_pre_mean_260_val'].transform(
    lambda x: x.fillna(x.mean()))
)

# Apply tolerance filter and create boolean masks
valid_uv_pre_mask_280 = (uv_pre_mask) & \
    (df_valid['UV_1_280_mAU'] >= df_valid['uv_pre_mean_280_val']
     * (1 - uv_tolerance)) & \
    (df_valid['UV_1_280_mAU'] <= df_valid['uv_pre_mean_280_val']
     * (1 + uv_tolerance))

valid_uv_pre_mask_260 = (uv_pre_mask) & \
    (df_valid['UV_2_260_mAU'] >= df_valid['uv_pre_mean_260_val']
     * (1 - uv_tolerance)) & \
    (df_valid['UV_2_260_mAU'] <= df_valid['uv_pre_mean_260_val']
     * (1 + uv_tolerance))

# Vectorized calculation for local_baseline_mean_280
# 1. Create column with values only where valid, else NaN
df_valid['UV_1_280_mAU_valid'] = df_valid['UV_1_280_mAU'].where(
    valid_uv_pre_mask_280)
# 2. Calculate mean of valid values per group
df_valid['local_baseline_mean_280'] = df_valid.groupby(['run_no', 'column'])['
    UV_1_280_mAU_valid'].transform('mean')

# Fallback 1: If NaN, use mean of volume_ml < 0 for that group
df_valid['UV_1_280_mAU_fallback'] = df_valid['UV_1_280_mAU'].where(uv_pre_mask)
fallback_mean_280 = df_valid.groupby(['run_no', 'column'])['UV_1_280_mAU_fallback']
    .transform('mean')
df_valid['local_baseline_mean_280'] = df_valid['local_baseline_mean_280'].fillna(
    fallback_mean_280)

# Fallback 2: If still NaN, use overall mean of the group
df_valid['local_baseline_mean_280'] = df_valid['local_baseline_mean_280'].fillna(
    df_valid.groupby(['run_no', 'column'])['UV_1_280_mAU'].transform('mean'))

# Vectorized calculation for local_baseline_mean_260
# 1. Create column with values only where valid, else NaN
df_valid['UV_2_260_mAU_valid'] = df_valid['UV_2_260_mAU'].where(
    valid_uv_pre_mask_260)
# 2. Calculate mean of valid values per group

```

```

df_valid['local_baseline_mean_260'] = df_valid.groupby(['run_no', 'column'])['UV_2_260_mAU_valid'].transform('mean')

# Fallback 1: If NaN, use mean of volume_ml < 0 for that group
df_valid['UV_2_260_mAU_fallback'] = df_valid['UV_2_260_mAU'].where(uv_pre_mask)
fallback_mean_260 = df_valid.groupby(['run_no', 'column'])['UV_2_260_mAU_fallback'].transform('mean')
df_valid['local_baseline_mean_260'] = df_valid['local_baseline_mean_260'].fillna(fallback_mean_260)

# Fallback 2: If still NaN, use overall mean of the group
df_valid['local_baseline_mean_260'] = df_valid['local_baseline_mean_260'].fillna(df_valid.groupby(['run_no', 'column'])['UV_2_260_mAU'].transform('mean'))

# Calculate SNR
df_valid['snr_280'] = (df_valid['peak_height_280'] - df_valid['local_baseline_mean_280']) / df_valid['noise_std_280']
df_valid['snr_260'] = (df_valid['peak_height_260'] - df_valid['local_baseline_mean_260']) / df_valid['noise_std_260']

# Step 9: Aggregate mean SNR and SD by column only
snr_280_agg = df_valid.groupby('column')['snr_280'].agg(['mean', 'std']).reset_index()
snr_260_agg = df_valid.groupby('column')['snr_260'].agg(['mean', 'std']).reset_index()

snr_agg = pd.merge(snr_280_agg, snr_260_agg, on='column', suffixes=('_280', '_260'))

# Step 10: Create visualizations
expected_columns = 4
unique_columns = snr_agg['column'].unique()
cols_to_plot = list(unique_columns)[:expected_columns]

fig, axes = plt.subplots(1, expected_columns, figsize=(6 * expected_columns, 6), sharey=True)
if expected_columns == 1:
    axes = [axes]

fig.suptitle('SNR Distribution by Column (4 Columns)', fontsize=16, fontweight='bold')

for i in range(expected_columns):
    ax = axes[i]
    if i < len(cols_to_plot):
        col_name = cols_to_plot[i]
        col_data = snr_agg[snr_agg['column'] == col_name]

        mean_280 = col_data['mean_280'].values[0]
        std_280 = col_data['std_280'].values[0]
        mean_260 = col_data['mean_260'].values[0]
        std_260 = col_data['std_260'].values[0]

        bar_width = 0.35
        ax.bar(i - bar_width/2, mean_280, yerr=std_280, width=bar_width, capsize=3,
               label='UV_1_280_mAU', alpha=0.7, color='skyblue')
        ax.bar(i + bar_width/2, mean_260, yerr=std_260, width=bar_width, capsize=3,

```



```

        label='UV_2_260_mAU', alpha=0.7, color='salmon')

    ax.set_xlabel(col_name)
    ax.set_ylabel('Mean_SNR')
    ax.set_title(f'Column:_{col_name}')
    ax.legend(loc='upper_right')
    ax.grid(axis='y', linestyle='--', alpha=0.3)
else:
    ax.text(0.5, 0.5, 'No_Data', transform=ax.transAxes, ha='center', va='
        center', fontsize=12)
    ax.set_xlabel(f'Column_{i+1}')
    ax.set_ylabel('Mean_SNR')
    ax.set_title(f'Column:_{i+1}_ (Missing)')
    ax.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.savefig('/workspace/snr_by_column_run.png')
plt.close()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
from scipy import stats
import os
import warnings

# File paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/conductivity_drift_analysis.md'

# Load data
df = pd.read_csv(input_file)

# Select required variables
required_vars = ['run_no', 'column', 'volume_ml', 'Conc_B_%', 'UV_1_280_mAU', '
    UV_2_260_mAU', 'Cond_mS_cm']
df = df[required_vars].dropna()

# Define signal columns
signal_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']

# Prepare output lines
output_lines = []

# Iterate over each column and signal
for col_name in df['column'].unique():
    col_data = df[df['column'] == col_name]

    for signal in signal_cols:
        # Validate Conc_B_% range before masking
        out_of_range_mask = (col_data['Conc_B_%'] < 0.0) | (col_data['Conc_B_%'] >
            100.0)
        if out_of_range_mask.any():
            warnings.warn(f"Column_{col_name}: Found Conc_B_% values outside 0-100
                range. Filtering them out.")
            col_data = col_data[~out_of_range_mask]

    if len(col_data) == 0:

```

```

        continue

# Pre-calculate Global Masks for Constant Gradient Regions
mask_0 = (col_data['Conc_B_%'] >= 0.0) & (col_data['Conc_B_%'] <= 0.5)
mask_100 = (col_data['Conc_B_%'] >= 99.5) & (col_data['Conc_B_%'] <=
100.0)
mask_const = mask_0 | mask_100

const_data = col_data[mask_const]

if len(const_data) < 10:
    continue

# Calculate drift rates (slope of signal vs volume_ml)
data_0 = const_data[mask_0]
data_100 = const_data[mask_100]

slopes = []

if len(data_0) >= 2:
    slope_0, _, _, _, _ = stats.linregress(data_0['volume_ml'], data_0[
        signal])
    slopes.append(slope_0)

if len(data_100) >= 2:
    slope_100, _, _, _, _ = stats.linregress(data_100['volume_ml'],
        data_100[signal])
    slopes.append(slope_100)

if not slopes:
    continue

# Prepare data for correlation
run_drifts = []
run_cond_means = []

# Refactor to Single Pass GroupBy Operation
# Replace Nested Python Loops with Vectorized GroupBy and Apply
for run_no, run_df in col_data.groupby('run_no'):
    # Filter constant gradient for this run using pre-calculated logic
    r_mask_0 = (run_df['Conc_B_%'] >= 0.0) & (run_df['Conc_B_%'] <= 0.5)
    r_mask_100 = (run_df['Conc_B_%'] >= 99.5) & (run_df['Conc_B_%'] <=
100.0)
    r_mask_const = r_mask_0 | r_mask_100

    r_const = run_df[r_mask_const]

    if len(r_const) < 10:
        continue

    r_data_0 = r_const[r_mask_0]
    r_data_100 = r_const[r_mask_100]

    r_slopes = []
    # Defensive check: ensure data is not empty before linregress
    if len(r_data_0) >= 2:
        s0, _, _, _, _ = stats.linregress(r_data_0['volume_ml'], r_data_0[
            signal])
        r_slopes.append(s0)

```

```

if len(r_data_100) >= 2:
    s100, _, _, _, _ = stats.linregress(r_data_100['volume_ml'],
    r_data_100[signal])
    r_slopes.append(s100)

if not r_slopes:
    continue

run_drift_rate = np.mean(r_slopes)

# Calculate mean Cond for the ENTIRE run (not just constant regions)
# Ensure Cond is positive for log transform
run_cond_vals = run_df['Cond_mS_cm']
if len(run_cond_vals) > 0:
    mean_cond = run_cond_vals.mean()
    # Ensure #run_cond_means# is only appended if #mean_cond > 0#
    if mean_cond > 0:
        log_cond = np.log(mean_cond)
        # Ensure #run_cond_means# is only appended if the run passes
        the length check (implied by reaching here)
        # and mean_cond > 0.
        run_drifts.append(run_drift_rate)
        run_cond_means.append(log_cond)

# Add a check to ensure #run_drifts# is not empty before calculating #
avg_drift#
if not run_drifts:
    continue

# Verify that #run_drifts# and #run_cond_means# have the same length
before calculating correlation
if len(run_drifts) != len(run_cond_means) or len(run_drifts) < 3:
    continue

# Calculate Pearson correlation
corr, p_val = stats.pearsonr(run_drifts, run_cond_means)

avg_drift = np.mean(run_drifts)

# Updated output string to include Avg_P-Value formatted to 4 decimal
places
output_line = f"Column_{col_name}_{Signal_{signal}}_Avg_Drift_Rate_(mAU/
mL):_{avg_drift:.6f}_{Avg_Correlation_With_Cond:_{corr:.4f}}_Avg_P-
Value:_{p_val:.4f}"
output_lines.append(output_line)

# Change the file open mode from 'w' to 'a' to strictly adhere to the Plan's
instruction to 'Append' to the file.
with open(output_file, 'a') as f:
    f.write('\n'.join(output_lines))

```

Listing 3: Code_3